

# Annexes

## SELECT DOCUMENT

```
SELECT
    Doc_Num, Doc_Type, Doc_Date,
    Tiers_Code, Doc_RS,
    Doc_RelReg, Doc_TTVAC, Doc_THTB,
    Doc_Timbre, Doc_TotalDPA,
    Doc_MargeTotalPourcent, Site_Code,
    Doc_DateSys
FROM biosphere.dbo.Document
WHERE Doc_DateSys >=
    ISNULL(
        (SELECT Last_Load_Date FROM ERP_STG.STG.CTRL_CHARGEMENT
         WHERE Table_Name = 'DOCUMENT'),
        '1900-01-01'
    );
```

## SELECT DOCUMENT\_D

```
SELECT
    DocD_ID, Doc_Num, Art_Code, Art_Des,
    DocD_UniteP, DocD_UniteC,
    DocD_Q, DocD_PUHTB,
    DocD_TxTVA, DocD_PUTTCB,
    DocD_THT, DocD_TVA, DocD_TTC,
    DocD_PUDPA, DocD_EMB,
    DocD_StockSfx, DocD_StockTn,
    DocD_Date
FROM biosphere.dbo.DocumentD
WHERE DocD_Date >= (
    SELECT Last_Load_Date
    FROM ERP_STG.STG.CTRL_CHARGEMENT
    WHERE Table_Name = 'DOCUMENT_D'
)
AND DocD_Date <= GETDATE
```

## SELECT DOCUMENT\_N

```
SELECT DocN_Code, DocN_Titre
FROM biosphere.dbo.DocumentN
```

SELECT STOJOU

```
SELECT StoJ_Id,  
       StoJ_Doc, Art_Code,  
       Stoj_Q,  
       StoJ_Prix,  
       StoJ_DateSys  
FROM biosphere.dbo.StoJou  
WHERE StoJ_DateSys >= (  
    SELECT Last_Load_Date  
    FROM ERP_STG.STG.CTRL_CHARGEMENT  
    WHERE Table_Name = 'STOJOU')  
AND StoJ_DateSys <= GETDATE()
```

SELECT TIERS

```
SELECT  
    Tiers_Code, Tiers_RS,  
    Tiers_Type, Tiers_Adresse,  
    Tiers_Ville, Tiers_Contact_Tel,  
    Tiers_Civilite, Tiers_Famille,  
    Date_Create  
FROM biosphere.dbo.Tiers  
WHERE Date_Create >= (  
    SELECT Last_Load_Date  
    FROM ERP_STG.STG.CTRL_CHARGEMENT  
    WHERE Table_Name = 'TIERS')  
AND Date_Create <= GETDATE()
```

## INSERT STG.DOCUMENT

```
const result = [];
const items = $input.all();

const clean = (val) => {
  if (val === null || val === undefined) return null;
  return String(val).replace(/'/g, "'");
};

const num = (val) => {
  if (val === null || val === undefined || val === '') return 0;
  return val;
};

const date = (val) => {
  if (!val) return "NULL";
  return "'" + val + "'";
};

const queries = [];

for (const item of items) {
  const d = item.json;

  // ignore rows without Doc_Num (because it will be PRIMARY KEY)
  if (!d.Doc_Num) continue;

  const query =
    "MERGE STG.DOCUMENT AS target " +
    "USING (VALUES ('" + clean(d.Doc_Num) + "')) AS source(Doc_Num) " +
    "ON target.Doc_Num = source.Doc_Num " +

    "WHEN MATCHED THEN UPDATE SET " +
    "Type = '" + clean(d.Doc_Type) + "', " +
    "Date = " + date(d.Doc_Date) + ", " +
    "Tiers.Code = '" + clean(d.Tiers.Code) + "', " +
    "Tiers.Num = N'" + clean(d.Doc_RS) + "', " +
    "Montant.Reg = " + num(d.Doc_RelReg) + ", " +
    "Total.IVA = " + num(d.Doc.ITVAC) + ", " +
    "Total.ht = " + num(d.Doc.THTB) + ", " +
    "Timbre = " + num(d.Doc.Timbre) + ", " +
    "Total.DPA = " + num(d.Doc.TotalDPA) + ", " +
    "TMarge.Pourcent = " + num(d.Doc.MargeTotalPourcent) + ", " +
    "Site = '" + clean(d.Site.Code) + "', " +
    "Last.Update = " + date(d.Doc.DateSVs) + " " +

    "WHEN NOT MATCHED THEN INSERT " +
    "(Doc_Num, Type, Date, Tiers.Code, Tiers.Num, Montant.Reg, Total.IVA, Total.ht, Timbre, Total.DPA, TMarge.Pourcent, Site, Last.Update) " +
    "VALUES (" +
    "'" + clean(d.Doc_Num) + "', " +
    "'" + clean(d.Doc_Type) + "', " +
    date(d.Doc_Date) + ", " +
    "'" + clean(d.Tiers.Code) + "', " +
    "N'" + clean(d.Doc_RS) + "', " +
    num(d.Doc_RelReg) + ", " +
    num(d.Doc.ITVAC) + ", " +
    num(d.Doc.THTB) + ", " +
    num(d.Doc.Timbre) + ", " +
    num(d.Doc.TotalDPA) + ", " +
    num(d.Doc.MargeTotalPourcent) + ", " +
    "'" + clean(d.Site.Code) + "', " +
    date(d.Doc.DateSVs) +
    ");";

  queries.push(query);
}

result.push({
  json: {
    query: queries.join(" ")
  }
});

return result;
```

## INSERT DOCUMNT\_D :

```
const result = [];
const items = $input.all();

const c = (v) => v == null ? '' : String(v).replace(/'/g, "'").replace(/\\/g, "\\").replace(/\r\n/g, ' ').trim();
const n = (v) => { const x = parseFloat(v); return isNaN(x) ? 0 : x; };
const d = (v) => {
  if (v == null || v === '') return 'NULL';
  return "" + String(v).replace('T', ' ').replace('Z', '').split('.')[0].replace(/'/g, "'") + "";
};

const values = items.map(item => {
  const x = item.json;
  return `('${c(x.DocD_ID)}', '${c(x.Doc_Num)}', '${c(x.Act_Code)}', N'${c(x.Act_Des)}', '${c(x.DocD_UniteP)}', ${n(x.DocD_Q)},
  ${n(x.DocD_PUHTB)}, ${n(x.DocD_TxTVA)}, ${n(x.DocD_PUTTCB)}, ${n(x.DocD_THI)}, ${n(x.DocD_TVA)}, ${n(x.DocD_TTC)}, ${n(x.DocD_PUDPA)},
  ${n(x.DocD_EMB)}, ${n(x.DocD_StockSfx)}, ${n(x.DocD_StockIn)}, ${d(x.DocD_Date)}, '${c(x.DocD_UniteC)}')`;
}).join(',');

const query = `MERGE STG.DOCUMENT_D AS target USING (VALUES ${values}) AS source
(DocD_ID, Doc_Num, Code_article, Designation, PIECES_COFFRET, Quantite, PUHT, TVA, PUTTC, MontantHT, TTVA, MontantTTC, PUA, Q_EMB,
StockSfx, StockIn, Last_Update, METH_stock) ON target.DocD_ID=source.DocD_ID WHEN MATCHED THEN UPDATE SET Doc_Num=source.Doc_Num, Code_article=
source.Code_article, Designation=source.Designation, PIECES_COFFRET=
source.PIECES_COFFRET, Quantite=source.Quantite, PUHT=source.PUHT, TVA=source.TVA, PUTTC=
source.PUTTC, MontantHT=source.MontantHT, TTVA=source.TTVA, MontantTTC=source.MontantTTC, PUA=source.PUA, Q_EMB=source.Q_EMB, StockSfx=source.StockSfx, StockIn=source.StockIn, Last_Update=
source.Last_Update, METH_stock=source.METH_stock WHEN NOT MATCHED THEN INSERT
(DocD_ID, Doc_Num, Code_article, Designation, PIECES_COFFRET, Quantite, PUHT, TVA, PUTTC, MontantHT, TTVA, MontantTTC, PUA, Q_EMB, StockSfx, StockIn, Last_Update, METH_stock) VALUES
(source.DocD_ID, source.Doc_Num, source.Code_article, source.Designation, source.PIECES_COFFRET, source.Quantite, source.PUHT, source.TVA, source.PUTTC, source.MontantHT,
source.TTVA, source.MontantTTC, source.PUA, source.Q_EMB, source.StockSfx, source.StockIn, source.Last_Update, source.METH_stock);`;

result.push({ json: { query } });
return result;
```

## INSERT DOCUMENT\_N :

```
const result = [];
const items = $input.all();
const clean = (val) => (val == null ? '' : String(val).replace(/'/g, "'"));

const mergeStatements = items.map(item => {
  const d = item.json;
  return "MERGE STG.DOCUMENT_N AS target " +
  "USING (SELECT '" + clean(d.DocN_Code) + "' AS Doc_CODE) AS source " +
  "ON target.Doc_CODE = source.Doc_CODE " +
  "WHEN MATCHED THEN UPDATE SET " +
  "Doc_Libelle = N'" + clean(d.DocN_Titre) + "' " +
  "WHEN NOT MATCHED THEN INSERT (Doc_CODE, Doc_Libelle) " +
  "VALUES ('" + clean(d.DocN_Code) + "', N'" + clean(d.DocN_Titre) + "');";
}).join('');

result.push({ json: { query: mergeStatements } });
return result;
```

## INSERT STOJOU :

```
const result = [];
const items = $input.all();
const c = (v) => v == null ? '' : String(v).replace(/'/g, "'").replace(/\\/g, "\\").replace(/\r\n/g, ' ').trim();
const n = (v) => { const x = parseFloat(v); return isNaN(x) ? 0 : x; };
const d = (v) => {
  if (v == null || v === '') return 'NULL';
  return "" + String(v).replace('T', ' ').replace('Z', '').split('.')[0].replace(/'/g, "'") + "";
};

const values = items.map(item => {
  const x = item.json;
  return `('${n(x.Stol_ID)}', '${c(x.Stol_Doc)}', '${c(x.Act_Code)}', ${d(x.Stol_DateSys)}, ${n(x.Stol_Q)}, ${n(x.Stol_Prix)})`;
}).join(',');

const query = `
SET IDENTITY_INSERT STG.STOJOU ON;

MERGE STG.STOJOU AS target
USING (VALUES ${values}) AS source (Stolou_ID, Document_ref, Act_Code, Date_Mouvement, Quantite, PUHT)
ON target.Stolou_ID = source.Stolou_ID
WHEN MATCHED THEN UPDATE SET
  Document_ref = source.Document_ref,
  Act_Code = source.Act_Code,
  Date_Mouvement = source.Date_Mouvement,
  Quantite = source.Quantite,
  PUHT = source.PUHT,
  Last_Update = GETDATE()
WHEN NOT MATCHED THEN INSERT (Stolou_ID, Document_ref, Act_Code, Date_Mouvement, Quantite, PUHT, Last_Update)
VALUES (source.Stolou_ID, source.Document_ref, source.Act_Code, source.Date_Mouvement, source.Quantite, source.PUHT, GETDATE());

SET IDENTITY_INSERT STG.STOJOU OFF;
`;

result.push({ json: { query } });
return result;
```

## INSERT TIERS :

```
const result = [];
const items = $input.all();
const c = (v) => v == null ? '' : String(v).replace(/'/g, "''").replace(/\\/g, "\\").replace(/\r\n/g, ' ').trim();
const d = (v) => {
  if (v == null || v === '') return 'NULL';
  return "" + String(v).replace('T', ' ').replace('Z', '').split('.')[0].replace(/'/g, "''") + "";
};

const values = items.map(item => {
  const x = item.json;
  return `('${c(x.Tiers_Code)}',N'${c(x.Tiers_RS)}', '${c(x.Tiers_Type)}',N'${c(x.Tiers_Adresse)}',N'${c(x.Tiers_Ville)}',
'${c(x.Tiers_Contact_Tel)}', '${c(x.Tiers_Civilite)}', '${c(x.Tiers_Famille)}', ${d(x.Date_Create)})`;
}).join(',');

const query = `
MERGE STG.TIERS AS target
USING (VALUES ${values}) AS source (Tiers_Code, Nom, Type, Adresse, Ville, Telephone, Civilite, Famille, Last_Update)
ON target.Tiers_Code = source.Tiers_Code
WHEN MATCHED THEN UPDATE SET
  Nom = source.Nom,
  Type = source.Type,
  Adresse = source.Adresse,
  Ville = source.Ville,
  Telephone = source.Telephone,
  Civilite = source.Civilite,
  Famille = source.Famille,
  Last_Update = source.Last_Update
WHEN NOT MATCHED THEN INSERT (Tiers_Code, Nom, Type, Adresse, Ville, Telephone, Civilite, Famille, Last_Update)
VALUES (source.Tiers_Code, source.Nom, source.Type, source.Adresse, source.Ville, source.Telephone, source.Civilite, source.Famille, source.Last_Update);
`;

result.push({ json: { query } });
return result;
```

## SELECT DIM\_CLIENT :

```
SELECT
  Tiers_Code,
  Nom,
  Adresse,
  Ville,
  Telephone AS Telephone,
  Civilite,
  Famille,
  Last_Update
FROM ERP_STG.STG.TIERS
WHERE Type = '01'
```

## SELECT DIM\_FRS :

```
SELECT
  Tiers_Code,
  Nom,
  Adresse,
  Ville,
  Telephone AS Telephone,
  Civilite,
  Famille,
  Last_Update
FROM ERP_STG.STG.TIERS
WHERE Type = '02'
```

SELECT DIM\_PRODUIT :

```
SELECT * FROM STG.ARTICLE
```

SELECT FACT\_ACHAT :

```
SELECT
    d.Doc_Num,
    d.Type,
    d.Montant_reg,
    d.Total_IVA,
    d.Total_ht,
    d.Site,
    dn.Code_article,
    dn.Designation,
    dn.PIECES_COFFRET,
    dn.Quantite,
    dn.PUHT,
    dn.TVA,
    dn.PUTTC,
    dn.MontantHT,
    dn.TTVA,
    dn.MontantTTC,
    dn.PUA,
    dn.Q_EMB,
    dn.StockSfx,
    dn.StockTn,
    dt.Date_Key,
    f.FRS_Key,
    p.Produit_Key
FROM STG.DOCUMENT_D dn
INNER JOIN STG.DOCUMENT d ON d.Doc_Num = dn.Doc_Num
INNER JOIN ERP_DWH.dbo.Dim_Date dt ON dt.Full_Date = d.Date
INNER JOIN ERP_DWH.dbo.Dim_FRS f ON f.Tiers_Code = d.Tiers_Code
INNER JOIN ERP_DWH.dbo.Dim_Produit p ON p.Code_Article = dn.Code_article
WHERE d.Type IN (
    'FACA', 'FACAF', 'FACAD', 'FACAI',
    'AVF', 'AVFI', 'AVFD', 'AVC', 'AVCT'
)
AND YEAR(d.Date) >= 2019
```

SELECT FACT\_VENTE :

```

SELECT
    dt.Date_Key,
    f.FRS_Key,
    p.Produit_Key,
    d.Doc_Num,
    d.Type,
    d.Total_ht AS Doc_Total,
    d.Montant_reg,
    d.Total_IVA,
    d.Total_ht,
    d.Site,
    dn.Designation,
    dn.PIECES_COFFRET,
    dn.Quantite,
    dn.PUHT,
    dn.IVA,
    dn.PUTTC,
    dn.MontantHT,
    dn.IVA,
    dn.MontantTTC,
    dn.PUA,
    dn.Q_EMB,
    dn.StockSfx,
    dn.StockTn
FROM STG.DOCUMENT d
INNER JOIN STG.DOCUMENT_D dn ON dn.Doc_Num = d.Doc_Num
INNER JOIN ERP_DWH.dbo.Dim_Date dt ON dt.Full_Date = d.Date
INNER JOIN ERP_DWH.dbo.Dim_FRS f ON f.Tiers_Code = d.Tiers_Code
INNER JOIN ERP_DWH.dbo.Dim_Produit p ON p.Code_Article = dn.Code_article
WHERE d.Type IN (
    'FACT','FACTU','FACTA','FACTAT',
    'FACTMS','FACTMT','FACTE','FACTEPF','ACL', 'RETT', 'RETEGT','BRSC', 'BREC'
)
AND YEAR(d.Date) >= 2019
AND d.Tiers_Code IN (
    SELECT Tiers_Code FROM ERP_DWH.dbo.Dim_FRS
)
AND dn.Code_article IN (
    SELECT Code_Article FROM ERP_DWH.dbo.Dim_Produit
)

```

SELECT FACT\_STOCK:

```

SELECT
    s.StoJou_ID,
    s.Document_ref,
    s.Art_Code,
    s.Date_Mouvement,
    s.Quantite,
    s.PUHT,
    dt.Date_Key,
    p.Produit_Key,
    CASE
        WHEN D.Type LIKE 'BE%' THEN 'ENTREE'
        WHEN D.Type LIKE 'BS%'
            OR D.Type LIKE 'BL%' THEN 'SORTIE'
        WHEN D.Type LIKE 'AV%' THEN 'RETOUR'
        WHEN D.Type IS NULL THEN 'INTERNE'
        ELSE 'AUTRE'
    END AS Type_Mouvement
FROM STG.STOJOU s
INNER JOIN ERP_DWH.dbo.Dim_Date dt
    ON dt.Full_Date = CAST(s.Date_Mouvement AS DATE)
INNER JOIN ERP_DWH.dbo.Dim_Produit p
    ON p.Code_Article = s.Art_Code
LEFT JOIN STG.DOCUMENT D
    ON s.Document_ref = D.Doc_Num
WHERE s.PUHT IS NOT NULL
    AND s.PUHT >= 0
    AND s.StoJou_ID IS NOT NULL;

```

## INSERT Dim\_Client :

```
const result = [];
const items = $input.all();
const c = (v) => v == null ? '' : String(v).replace(/'/g, "''").replace(/\\/g, "\\\").replace(/[\\r\\n]/g, ' ').trim();
const d = (v) => {
  if (v == null || v === '') return 'NULL';
  return "" + String(v).replace('T', ' ').replace('Z', '').split('.')[0].replace(/'/g, "''") + "";
};

const values = items.map(item => {
  const x = item.json;
  return `('${c(x.Tiers_Code)}',N'${c(x.Nom)}',N'${c(x.Adresse)}',N'${c(x.Ville)}', '${c(x.Telephone)}', '${c(x.Civilite)}', '${c(x.Famille)}', ${d(x.Last_Update)})`;
}).join(',');

const query = `
MERGE ERP_DWH.dbo.Dim_Client AS target
USING (VALUES ${values}) AS source (Tiers_Code, Nom, Adresse, Ville, Telephone, Civilite, Famille, Last_Update)
ON target.Tiers_Code = source.Tiers_Code
WHEN MATCHED THEN UPDATE SET
  Nom = source.Nom,
  Adresse = source.Adresse,
  Ville = source.Ville,
  Telephone = source.Telephone,
  Civilite = source.Civilite,
  Famille = source.Famille,
  Last_Update = source.Last_Update
WHEN NOT MATCHED THEN INSERT (Tiers_Code, Nom, Adresse, Ville, Telephone, Civilite, Famille, Last_Update)
VALUES (source.Tiers_Code, source.Nom, source.Adresse, source.Ville, source.Telephone, source.Civilite, source.Famille, source.Last_Update);
`;

result.push({ json: { query } });
return result;
```

## INSERT Dim\_FRS :

```
const result = [];
const items = $input.all();
const c = (v) => v == null ? '' : String(v).replace(/'/g, "''").replace(/\\/g, "\\\").replace(/[\\r\\n]/g, ' ').trim();
const d = (v) => {
  if (v == null || v === '') return 'NULL';
  return "" + String(v).replace('T', ' ').replace('Z', '').split('.')[0].replace(/'/g, "''") + "";
};

const values = items.map(item => {
  const x = item.json;
  return `('${c(x.Tiers_Code)}',N'${c(x.Nom)}',N'${c(x.Adresse)}',N'${c(x.Ville)}', '${c(x.Telephone)}', '${c(x.Civilite)}', '${c(x.Famille)}', ${d(x.Last_Update)})`;
}).join(',');

const query = `
MERGE ERP_DWH.dbo.Dim_FRS AS target
USING (VALUES ${values}) AS source (Tiers_Code, Nom, Adresse, Ville, Telephone, Civilite, Famille, Last_Update)
ON target.Tiers_Code = source.Tiers_Code
WHEN MATCHED THEN UPDATE SET
  Nom = source.Nom,
  Adresse = source.Adresse,
  Ville = source.Ville,
  Telephone = source.Telephone,
  Civilite = source.Civilite,
  Famille = source.Famille,
  Last_Update = source.Last_Update
WHEN NOT MATCHED THEN INSERT (Tiers_Code, Nom, Adresse, Ville, Telephone, Civilite, Famille, Last_Update)
VALUES (source.Tiers_Code, source.Nom, source.Adresse, source.Ville, source.Telephone, source.Civilite, source.Famille, source.Last_Update);
`;

result.push({ json: { query } });
return result;
```



## INSERT Dim\_Produit :

```

const result = [];
const items = $input.all();
const c = (v) => v == null ? '' : String(v)
    .replace(/'/g, "'")
    .replace(/\\/g, "\\")
    .replace(/\r\n/g, ' ');
const d = (v) => {
    if (v == null || v === '') return 'NULL';
    return "" + String(v)
        .replace('T', ' ')
        .replace('Z', '')
        .split('.')[0]
        .replace(/'/g, "'") + " ";
};
const n = (v) => v == null || v === '' ? 'NULL' : v;
const values = items.map(item => {
    const x = item.json;
    return `
        '${c(x.Code_Article)}',N'${c(x.Designation)}',N'${c(x.Categorie)}',N'${c(x.Famille1)}',
        N'${c(x.Famille2)}',N'${c(x.Famille3)}',N'${c(x.Unite)}',${n(x.TVA)},${n(x.GesStock)},${n(x.GesLot)},
        ${n(x.Stock)},${n(x.PMP)},${n(x.DPA)},${n(x.PV)},N'${c(x.Stat1)}',N'${c(x.Stat5)}',${d(x.Last_Update)}
    `;
});
const query = `
MERGE ERP_DWH.dbo.Dim_Produit AS target
USING (VALUES ${values}) AS source (Code_Article,Designation,Categorie,Famille1,Famille2,Famille3,Unite,TVA,GesStock,GesLot,Stock,PMP,DPA,PV,Stat1,Stat5,
Last_Update
)
ON target.Code_Article = source.Code_Article
WHEN MATCHED THEN UPDATE SET
    Designation = source.Designation,Categorie = source.Categorie,
    Famille1 = source.Famille1,Famille2 = source.Famille2,Famille3 = source.Famille3,
    Unite = source.Unite,TVA = source.TVA,GesStock = source.GesStock,
    GesLot = source.GesLot,Stock = source.Stock,PMP = source.PMP,DPA = source.DPA,
    PV = source.PV,Stat1 = source.Stat1,Stat5 = source.Stat5,Last_Update = source.Last_Update
WHEN NOT MATCHED THEN INSERT (
    Code_Article,Designation,Categorie,Famille1,Famille2,Famille3,
    Unite,TVA,GesStock,GesLot,Stock,PMP,DPA,
    PV,Stat1,Stat5,
    Last_Update
)
VALUES (source.Code_Article,source.Designation,source.Categorie,
source.Famille1,source.Famille2,source.Famille3,source.Unite,
source.TVA,source.GesStock,source.GesLot,source.Stock,
source.PMP,source.DPA,source.PV,source.Stat1,source.Stat5,source.Last_Update
);
`;
result.push({ json: { query } });
return result;

```

## INSERT FACT\_ACHAT :

```
const result = [];
const items = $input.all();
const c = (v) => v == null ? '' : String(v).replace(/'/g, '"').replace(/\\/g, '\\\\').replace(/[\r\n]/g, ' ').trim();
const n = (v) => { const x = parseFloat(v); return isNaN(x) ? 0 : x; };
const values = items.map(item => {
  const x = item.json;
  return `(${n(x.Date_Key)},${n(x.Produit_Key)},${n(x.FRS_Key)},'${c(x.Doc_Num)}','${c(x.Type)}',${n(x.Total_ht)},${n(x.Montant_reg)},
${n(x.Total_TVA)},${n(x.Total_ht)},'${c(x.Site)}',N'${c(x.Designation)}','${c(x.PIECES_COFFRET)}',${n(x.Quantite)},${n(x.PUHT)},${n(x.TVA)},
${n(x.PUTTC)},${n(x.MontantHT)},${n(x.TTVA)},${n(x.MontantTIC)},${n(x.PUA)},${n(x.Q_EMB)},${n(x.StockSfx)},${n(x.StockIn)})`;
}).join(',');
const query = `
INSERT INTO ERP_DWH.dbo.Fact_Achat
(Date_Key, Produit_Key, FRS_Key, Doc_Num, Type, Doc_Total, Montant_reg,
Total_TVA, Total_ht, Site, Designation, PIECES_COFFRET, Quantite, PUHT,
TVA, PUTTC, MontantHT, TTVA, MontantTIC, PUA, Q_EMB, StockSfx, StockIn)
SELECT DISTINCT
Date_Key, Produit_Key, FRS_Key, Doc_Num, Type, Doc_Total, Montant_reg,
Total_TVA, Total_ht, Site, Designation, PIECES_COFFRET, Quantite, PUHT,
TVA, PUTTC, MontantHT, TTVA, MontantTIC, PUA, Q_EMB, StockSfx, StockIn
FROM (VALUES ${values}) AS source
(Date_Key, Produit_Key, FRS_Key, Doc_Num, Type, Doc_Total, Montant_reg,
Total_TVA, Total_ht, Site, Designation, PIECES_COFFRET, Quantite, PUHT,
TVA, PUTTC, MontantHT, TTVA, MontantTIC, PUA, Q_EMB, StockSfx, StockIn)
WHERE NOT EXISTS (
  SELECT 1 FROM ERP_DWH.dbo.Fact_Achat t
  WHERE t.Doc_Num = source.Doc_Num
  AND t.Produit_Key = source.Produit_Key
  AND t.Date_Key = source.Date_Key
  AND t.FRS_Key = source.FRS_Key
);
`;
result.push({ json: { query } });
return result;
```

## INSERT FACT\_VENTE :

```
const result = [];
const items = $input.all();
const c = (v) => {
  if (v == null) return '';
  return String(v)
    .replace(/\\/g, '\\\\')
    .replace(/'/g, '"')
    .replace(/[\r\n\t]/g, ' ')
    .replace(/[\x00-\x1F\x7F]/g, '')
    .trim();
};
const n = (v) => { const x = parseFloat(v); return isNaN(x) ? 0 : x; };
const values = items.map(item => {
  const x = item.json;
  return `(${n(x.Date_Key)},${n(x.Produit_Key)},${n(x.Client_Key)},'${c(x.Doc_Num)}','${c(x.Type)}',${n(x.Total_ht)},
${n(x.Montant_reg)},${n(x.Total_TVA)},${n(x.Total_ht)},${n(x.Timbre)},${n(x.Total_DPA)},${n(x.IMarge_Pourcent)},'${c(x.Site)}',
N'${c(x.Designation)}','${c(x.PIECES_COFFRET)}',${n(x.Quantite)},${n(x.PUHT)},${n(x.TVA)},${n(x.PUTTC)},${n(x.MontantHT)},${n(x.TTVA)},
${n(x.MontantTIC)},${n(x.PUA)},${n(x.Q_EMB)},${n(x.StockSfx)},${n(x.StockIn)})`;
}).join(',');
const query = `
INSERT INTO ERP_DWH.dbo.Fact_Vente
(Date_Key, Produit_Key, Client_Key, Doc_Num, Type, Doc_Total, Montant_reg,
Total_TVA, Total_ht, Timbre, Total_DPA, IMarge_Pourcent, Site, Designation,
PIECES_COFFRET, Quantite, PUHT, TVA, PUTTC, MontantHT, TTVA, MontantTIC,
PUA, Q_EMB, StockSfx, StockIn)
VALUES ${values}
`;
result.push({ json: { query } });
return result;
```

## Extrait code : Graphique comportements contrastés :

```
# ===== GRAPHIQUE 1 : COMPORTEMENTS CONTRASTÉS =====
print("\n" + "=" * 80)
print("GRAPHIQUE 1 : Comportements temporels contrastés")
print("=" * 80)

fig, ax = plt.subplots(figsize=(14, 7))

colors = ['#2ecc71', '#e74c3c', '#f39c12']
produits_selected = [produit_stable, produit_volatile, produit_ruptures]
labels_selected = [
    f'Produit stable\n({produit_stable[:15]}...)',
    f'Produit volatile\n({produit_volatile[:15]}...)',
    f'Produit ruptures\n({produit_ruptures[:15]}...)'
]

for produit, color, label in zip(produits_selected, colors, labels_selected):
    data = df_agg[df_agg['Code_Article'] == produit].copy()
    data = data.sort_values('Full_Date')
    ax.plot(data['Full_Date'], data['Stock'],
            marker='o', linewidth=2.5, markersize=3,
            label=label, color=color, alpha=0.8)

ax.set_xlabel('Date', fontsize=12, fontweight='bold')
ax.set_ylabel('Niveau de Stock (unités)', fontsize=12, fontweight='bold')
ax.set_title('Figure 1 : Comportements temporels contrastés des niveaux de stock selon les produits',
            fontsize=14, fontweight='bold', pad=20)
ax.legend(fontsize=10, loc='best', framealpha=0.95)
ax.grid(True, alpha=0.3)
ax.tick_params(axis='both', labelsize=10)
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.savefig('/mnt/user-data/outputs/Fig1_Comportements_Contrastes.png', dpi=300, bbox_inches='tight')
print("✓ Sauvegardé : Fig1_Comportements_Contrastes.png")
plt.close()
```

## Extrait code : Graphique ruptures récurrentes :

```
# ===== GRAPHIQUE 2 : RUPTURES RÉCURRENTES =====
print("\n" + "=" * 80)
print("GRAPHIQUE 2 : Ruptures récurrentes")
print("=" * 80)

fig, ax = plt.subplots(figsize=(14, 7))

data_ruptures = df_agg[df_agg['Code_Article'] == produit_ruptures].copy()
data_ruptures = data_ruptures.sort_values('Full_Date')

# Courbe de stock
ax.plot(data_ruptures['Full_Date'], data_ruptures['Stock'],
        marker='o', linewidth=2.5, markersize=4,
        label='Niveau de stock', color='#3498db', zorder=3)

# Points de rupture (stock = 0)
ruptures = data_ruptures[data_ruptures['Stock'] == 0]
if len(ruptures) > 0:
    ax.scatter(ruptures['Full_Date'], ruptures['Stock'],
               color='#e74c3c', s=150, marker='x', zorder=4,
               label=f'Ruptures (n={len(ruptures)})', edgecolors='darkred', linewidth=2)
else:
    ax.scatter([], [], color='#e74c3c', s=150, marker='x', zorder=4,
               label='Ruptures (n=0)', edgecolors='darkred', linewidth=2)

# Ligne de seuil à 0
ax.axhline(y=0, color='#e74c3c', linestyle='--', linewidth=2, alpha=0.7, label='Seuil rupture (0)')

ax.set_xlabel('Date', fontsize=12, fontweight='bold')
ax.set_ylabel('Niveau de Stock (unités)', fontsize=12, fontweight='bold')
ax.set_title(f'Figure 2 : Exemple de ruptures récurrentes observées dans une série temporelle de stock\n({produit_ruptures[:40]}',
            fontsize=14, fontweight='bold', pad=20)
ax.legend(fontsize=11, loc='best', framealpha=0.95)
ax.grid(True, alpha=0.3)
ax.tick_params(axis='both', labelsize=10)
plt.xticks(rotation=45, ha='right')
```

## Extrait code : Graphique autocorrélation :

```
# ===== GRAPHIQUE 3 : AUTOCORRÉLATION =====
print("\n" + "=" * 80)
print("GRAPHIQUE 3 : Autocorrélation")
print("=" * 80)

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# ACF pour les 3 produits
labels_short = ['Produit stable', 'Produit volatile', 'Produit ruptures']

for idx, (produit, color, label_short) in enumerate(zip(produits_selected, colors, labels_short)):
    data = df_agg[df_agg['Code_Article'] == produit]['Stock'].values

    if len(data) > 30:
        ax = axes[idx]
        plot_acf(data, lags=30, ax=ax, color=color,
                  marker='o', markersize=6, linewidth=2)

        ax.set_title(f'ACF - {label_short}',
                     fontsize=11, fontweight='bold')
        ax.set_xlabel('Retard (Lag)', fontsize=10)
        ax.set_ylabel('Autocorrélation', fontsize=10)
        ax.grid(True, alpha=0.3)
        ax.axhline(y=0, color='black', linestyle='-', linewidth=0.8)

fig.suptitle('Figure 3 : Fonction d\'autocorrélation des niveaux de stock',
             fontsize=14, fontweight='bold', y=1.00)

plt.tight_layout()
plt.savefig('/mnt/user-data/outputs/Fig3_Autocorrelation.png', dpi=300, bbox_inches='tight')
print("✓ Sauvegardé : Fig3_Autocorrelation.png")
plt.close()
```